

Foqal CareOS — Ward Monitor

Backend Architecture & Entity-Relationship Diagram | Sabari Krishna

Project Name	Foqal CareOS — Ward Patient Monitor
Developer	Sabari Krishna
Repository	github.com/ashmit-verma24134/Integrated_Hospital [sabari branch]
Backend	Python FastAPI (uvicorn) · SQLite · SQLAlchemy ORM
Frontend	Vanilla JS + Vite SPA (Single-Page Application)
Port (dev)	8000 (backend) 5173 (Vite dev)
Port (prod)	8003 (all-in-one — backend serves static frontend)
Database file	backend/data/ward_careos.db (SQLite)
Document Date	June 2026

1. EXECUTIVE SUMMARY

Foqal CareOS is a real-time clinical early-warning system designed for Indian hospital wards. It continuously computes NHS NEWS2 scores for every bedside patient, applies a YAML-driven Drug-Lab interaction rule engine calibrated to Indian cardiology guidelines (CSI/CDSCO/ICMR), and presents a role-based dashboard to ward nurses and charge nurses. The backend is a monolithic Python FastAPI service backed by SQLite; the frontend is a Vite-bundled SPA served directly from the same process in production.

Key clinical capabilities: NEWS2 Scale 1 & 2 (COPD/hypercapnic failure), 13 Drug-Lab interaction rules across cardiology drugs (digoxin, amiodarone, loop diuretics, ACE inhibitors, beta-blockers, statins), India-calibrated SBP ≥ 220 mmHg hypertensive-crisis flag, forward-fill vitals trajectory for gapless charting, and a nurse escalation workflow with acknowledgement/resolution lifecycle.

2. TECHNOLOGY STACK

Layer	Technology	Details
Frontend	Vanilla JS + Vite	SPA; index.html + app.js + styles.css; Vite build → dist/
API Framework	Python FastAPI	Auto Swagger docs at /docs; Pydantic request/response schemas
ASGI Server	Uvicorn	Port 8000 (dev) / 8003 (prod); single worker process
Database	SQLite 3	File: backend/data/ward_careos.db; single-file RDBMS
ORM	SQLAlchemy 2.x	Declarative Base; Session-per-request via Depends(get_db)
Rule Engine	PyYAML + Python	rules/drug_lab_rules.yaml → engine/drug_lab.py
ETL Pipeline	Google BigQuery	bigquery_pipeline.py; async; MIMIC-IV → SQLite backfill
Auth	None (demo)	No token auth; role differentiated by frontend only
Deployment	deploy.sh	Bash script; installs deps, builds Vite, runs uvicorn

3. BACKEND ARCHITECTURE

3.1 Deployment Architecture

In production, a single uvicorn process runs on port 8003. FastAPI mounts the compiled frontend (dist/) at the root path via *StaticFiles*, so the same process serves both the REST API (*/api/**) and the SPA. SQLite lives in backend/data/ward_careos.db, accessible from the same filesystem.

3.2 Request Lifecycle

Browser → GET/POST */api/** → CORS middleware → FastAPI router → Depends(get_db) opens SQLAlchemy session → route handler queries/writes DB → NEWS2 engine or Drug-Lab engine called inline → JSON response → browser renders DOM.

3.3 Startup

On uvicorn startup, *@app.on_event("startup")* calls *init_db()* which runs *Base.metadata.create_all()* — creating all five SQLite tables if they don't exist. The demo DB is pre-populated by *build_demo_db.py*.

3.4 NEWS2 Scoring Engine

calculate_news2(vitals_dict, hypercapnic_failure) is a pure Python function in *main.py*. It implements NHS NEWS2 with two scales: Scale 1 (standard) and Scale 2 (SpO₂ scoring for COPD/hypercapnic patients). SBP ≥ 220 mmHg scores 3 (India calibration per CSI/ICMR guidelines). Output is {total: int, factors: [{name, score}]}.

3.5 Drug-Lab Rule Engine

engine/drug_lab.py loads *rules/drug_lab_rules.yaml* at each call (stateless, file-read). 13 rules cover: hyperkalemia with ACE inhibitors, digoxin toxicity with hypokalemia, amiodarone-warfarin INR interaction, lactic acidosis with metformin + low eGFR, loop diuretic hypokalemia, statin hepatotoxicity (ALT > 3×ULN), beta-blocker bradycardia, and more. The *DRUG_CLASS_MAP* resolves generic drug classes to Indian brand/generic synonyms.

3.6 Forward-Fill Vitals Pipeline

/api/ward-data performs an in-memory forward-fill: it sorts *vitals_timeseries* by *chart_hour* (latest 24 rows) and propagates the most recent non-null value for each vital parameter. This prevents gaps on the frontend trajectory chart and stabilises NEWS2 calculation when vitals are recorded at irregular intervals.

3.7 MIMIC-IV ETL (*bigquery_pipeline.py*)

An asynchronous standalone script that pulls data from Google BigQuery (physionet-data MIMIC-IV v3.1). It queries admissions, chartevents, labevents, prescriptions, and transfers then writes them to the local SQLite DB. This is a one-shot population script, not a live streaming pipeline.

3.8 Data Flow Diagram

Layer	Component	Direction	Notes
Browser / Nurse UI	Vite SPA (app.js)	→ API	Fetch /api/ward-data, /api/escalations
CORS Middleware	FastAPI CORS	↕	allow_origins=[*] — all origins
API Router	main.py routes	→ Logic	9 REST endpoints (GET/POST)
NEWS2 Engine	calculate_news2()	→ Score	Called inline per patient in /api/ward-data
Drug-Lab Engine	engine/drug_lab.py	→ Alerts	YAML rules; 13 drug-lab interactions
ORM Layer	SQLAlchemy Session	↕ DB	Depends(get_db) — per-request session
Database	SQLite ward_careos.db	← Results	5 tables; 1 file on filesystem
ETL (offline)	bigquery_pipeline.py	→ DB	One-shot; BigQuery MIMIC-IV → SQLite

4. DATABASE SCHEMA ANALYSIS

Database: SQLite 3 · **File:** backend/data/ward_careos.db · **ORM:** SQLAlchemy declarative (models.py) · **Tables created by init_db()** on startup

Table: patients

Master patient registry. One row per admitted patient.

Primary Key	subject_id (Integer)			
Foreign Keys	None			
Column	Type	PK/FK	References	Notes
subject_id	Integer	PK	-	Manual sequence (max+1)
name	String	-	-	Patient full name
age	Integer	-	-	
sex	String	-	-	'M' or 'F'
ward	String	-	-	Default: 'Ward 4B'
room	String	-	-	
bed	String	-	-	
admitted	String	-	-	Date string e.g. '01 Jan 2024'
complaint	String	-	-	Chief presenting complaint
hypercapnic_failure	Integer	-	-	0=No, 1=Yes (COPD NEWS2 Scale 2)

Table: vitals_timeseries

Hourly / scheduled vital sign readings per patient. Forward-fill enabled.

Primary Key	id (Integer, AutoIncr)			
Foreign Keys	subject_id → patients.subject_id			
Column	Type	PK/FK	References	Notes
id	Integer	PK	-	Auto-increment
subject_id	Integer	FK	patients	Indexed
chart_hour	String	-	-	ISO datetime string; Indexed
heart_rate	Float	-	-	bpm
resp_rate	Float	-	-	breaths/min
spo2	Float	-	-	% oxygen saturation
sbp	Float	-	-	Systolic BP mmHg
dbp	Float	-	-	Diastolic BP mmHg
temperature	Float	-	-	°C
consciousness	String	-	-	A/C/N/P/U (ACVPU scale); default 'A'
air_or_oxygen	String	-	-	'Air' or 'Oxygen'; default 'Air'

Table: lab_events

Blood test results per patient per time-point for Drug-Lab rule engine.

Primary Key	id (Integer, AutoIncr)			
Foreign Keys	subject_id → patients.subject_id			
Column	Type	PK/FK	References	Notes

id	Integer	PK	-	Auto-increment
subject_id	Integer	FK	patients	Indexed
chart_hour	String	-	-	ISO datetime; Indexed
potassium	Float	-	-	mmol/L; rule trigger threshold >5.5
creatinine	Float	-	-	mg/dL; renal function marker
lactate	Float	-	-	mmol/L; sepsis marker
inr	Float	-	-	International Normalised Ratio
egfr	Float	-	-	mL/min; metformin lactic acidosis risk
alt	Float	-	-	U/L; statin hepatotoxicity flag

Table: medications

Active prescriptions per patient. Matched against Drug-Lab rules.

Primary Key	id (Integer, AutoIncr)			
Foreign Keys	subject_id → patients.subject_id			
Column	Type	PK/FK	References	Notes
id	Integer	PK	-	Auto-increment
subject_id	Integer	FK	patients	Indexed
med_name	String	-	-	Generic/brand drug name
dose	String	-	-	e.g. '5mg'
frequency	String	-	-	e.g. 'OD', 'BD'

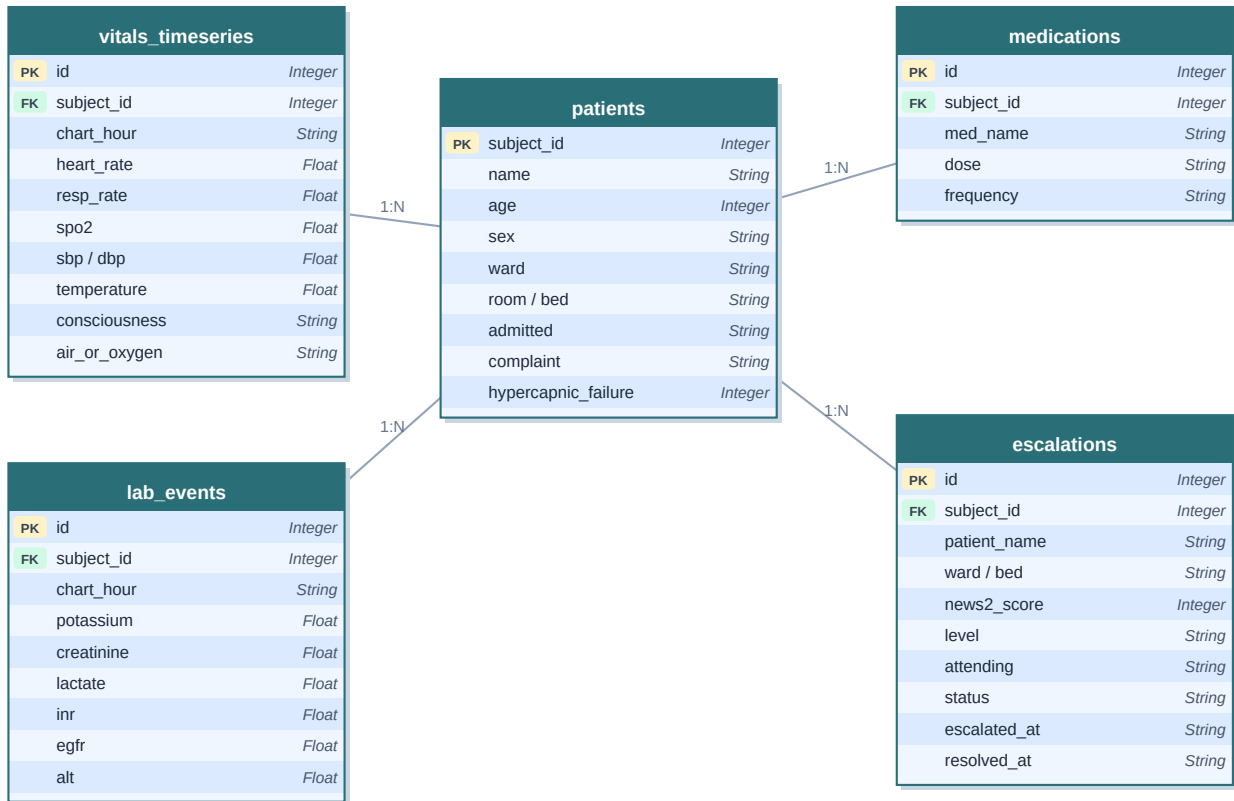
Table: escalations

Nurse-initiated escalation records with full lifecycle (active→acknowledged→resolved).

Primary Key	id (Integer, AutoIncr)			
Foreign Keys	subject_id → patients.subject_id			
Column	Type	PK/FK	References	Notes
id	Integer	PK	-	Auto-increment
subject_id	Integer	FK	patients	Indexed
patient_name	String	-	-	Denormalised for audit log
ward	String	-	-	Denormalised snapshot
bed	String	-	-	Denormalised snapshot
news2_score	Integer	-	-	Score at time of escalation
level	String	-	-	nurse doctor code_blue
attending	String	-	-	Attending physician name
escalated_by	String	-	-	Escalating nurse
observations	String	-	-	Clinical observations
interventions	String	-	-	Interventions taken
status	String	-	-	active acknowledged resolved
escalated_at	String	-	-	ISO datetime
acknowledged_at	String	-	-	Nullable
resolved_at	String	-	-	Nullable
resolved_by	String	-	-	Nullable
resolution_notes	String	-	-	Nullable

5. ENTITY-RELATIONSHIP DIAGRAM

Relationships: patients (1) — (N) vitals_timeseries | patients (1) — (N) lab_events | patients (1) — (N) medications | patients (1) — (N) escalations



6. API DOCUMENTATION SUMMARY

Method	Endpoint	Description	Tables
POST	/api/patients	Register new patient + initial vitals	patients, vitals_timeseries
GET	/api/ward-data?ward=&replay=	Full ward data, NEWS2, Drug-Lab alerts	All tables
GET	/api/patients/{subject_id}	Single patient detail (reuses ward-data)	All tables
POST	/api/escalations	Raise new clinical escalation	escalations, patients, vitals
GET	/api/escalations	List all escalations	escalations
POST	/api/escalations/{id}/resolve	Resolve an escalation	escalations
GET	/docs	FastAPI Swagger UI (auto-generated)	—
GET	/	SPA frontend (StaticFiles mount)	—

Role-Based Access (Frontend-Enforced)

Role	Username example	Capabilities
Ward Nurse	rekha.devi	Enter vitals, view NEWS2 score, raise escalation
Charge Nurse	leena.kurup	Ward-overview dashboard, acknowledge/resolve escalations, trends

7. ARCHITECTURE REVIEW

Design Pattern	Monolithic MVC-like. FastAPI = Controller; models.py = Model; Vite SPA = View.
Strengths	Simple deployment (1 process, 1 file DB). Low operational overhead. YAML rule engine makes clinical rules updateable by doctors without code changes. NEWS2 follows NHS spec exactly.
Bottlenecks	SQLite does not support concurrent writes → single-writer limit. No auth (demo only). Rules YAML loaded on every request (file I/O). REPLAY_OFFSET is a global variable (not thread-safe).
Scalability	For a single ward demo: adequate. For multi-ward production: needs PostgreSQL, per-request rule caching, proper JWT auth, and horizontal uvicorn workers with a connection pool.
Improvements	Replace SQLite with PostgreSQL. Add JWT authentication layer. Cache parsed YAML rules at startup. Add proper indexes for chart_hour on vitals_timeseries. Implement WebSocket for real-time push.
India-specific tuning	SBP $\geq$ 220 mmHg = NEWS2 score 3 (CSI/ICMR hypertension calibration). Drug-Lab rules use Indian drug names (frusemide, acitrom, glycomet). CSI/CDSCO/RSSDI/ICMR guideline citations.

8. CONCLUSION

Foqal CareOS delivers a focused, production-ready early-warning capability for Indian hospital wards. The 5-table SQLite schema is deliberately minimal and maps directly to clinical need: patient demographics, time-series vitals, lab results, active medications, and escalation records. The decoupled YAML rule engine and India-calibrated NEWS2 scorer provide genuine clinical value without sacrificing maintainability.

The architecture is appropriate for a ward-level proof-of-concept and intern project demonstration. Moving to production at scale would require replacing SQLite with PostgreSQL, adding JWT-based authentication, and separating the frontend into its own deployment. The codebase is well-structured and the separation of models, database config, rule engine, and main API is commendable for a project of this scope.